

# Churn Model — Training Methodology (Study Guide)

A step-by-step explanation of **how** the churn model is trained, **what type of training** is used, and **which model** is used — written for study purposes. All references are to the actual implementation in `scripts/train_models.py`.

## 1. Problem Framing

We solve a **supervised binary classification** problem:

- **Input** ( $X$ ): a vector of 64 point-in-time customer features (age, tenure, transaction counts, balances, engagement, product holdings, ...).
- **Target** ( $y$ ): whether the customer **churns** — defined as their `rel_customer_status` being a positive value ("Closed"), i.e.  $y \in \{0, 1\}$ .
- **Output**: a probability  $p(\text{churn}) \in [0, 1]$  per customer.

Because churn is rare ( $\approx 5.3\%$  of customers), this is an **imbalanced** binary classification problem, which drives several design choices below.

## 2. The Model: XGBoost

### 2.1 What XGBoost is

**XGBoost** (eXtreme Gradient Boosting) is an ensemble of **gradient-boosted decision trees**. Instead of one big model, it builds many small, weak trees sequentially; each new tree is fitted to correct the **residual errors** of the combined ensemble so far.

The final prediction is the sum of all tree outputs passed through a sigmoid:

$$\hat{y}_i = \sigma \left( \sum_{k=1}^K f_k(x_i) \right), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

### 2.2 How it learns (per boosting round)

Each round  $k$  adds a tree  $f_k$  that minimises the **logistic loss**:

$$\mathcal{L} = -\frac{1}{N} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

using gradient descent **in function space** (not weight space). This is why the console prints a line per round — `validation_0-auc / validation_1-auc / logloss` — showing how the ensemble improves after each added tree.

### 2.3 Hyperparameters used (and what each does)

| Hyperparameter                | Value                           | Meaning                                               |
|-------------------------------|---------------------------------|-------------------------------------------------------|
| <code>objective</code>        | <code>binary:logistic</code>    | Logistic loss → outputs a probability via sigmoid     |
| <code>eval_metric</code>      | <code>["auc", "logloss"]</code> | Metrics tracked each round during training            |
| <code>n_estimators</code>     | 50                              | Number of boosting rounds (trees)                     |
| <code>learning_rate</code>    | 0.05                            | Shrinkage — how much each tree contributes            |
| <code>max_depth</code>        | 4                               | Max tree depth — shallow trees = less overfit         |
| <code>min_child_weight</code> | 2                               | Min sum of instance weights per leaf (regularisation) |
| <code>reg_lambda</code>       | 1.0                             | L2 regularisation on leaf weights                     |
| <code>subsample</code>        | 0.8                             | Fraction of <b>rows</b> sampled per tree              |
| <code>colsample_bytree</code> | 0.8                             | Fraction of <b>columns</b> sampled per tree           |
| <code>scale_pos_weight</code> | ~18.0 (auto)                    | Class-imbalance weighting (see §3.2)                  |
| <code>random_state</code>     | 42                              | Reproducibility                                       |

**Why these values:** `max_depth=4` + `reg_lambda=1.0` + `min_child_weight=2` keep the model simple to control the overfit gap; `subsample/colsample_bytree` add stochasticity (each tree sees a different sample) which reduces variance.

## 3. Type of Training

### 3.1 Supervised, offline, full-batch

- **Supervised:** every training row has a known label.
- **Offline:** trained as a batch job (`scripts/train_models.py`), not online.
- **Full-batch:** the whole training set is used each round (no mini-batching — the dataset fits in memory).

### 3.2 Class-weight balancing

With 5.3% positives, a default model would trivially predict "no churn". XGBoost is given a **scale\_pos\_weight**:

$$\text{scale\_pos\_weight} = \frac{N_{\text{negative}}}{N_{\text{positive}}} = \frac{9470}{526} \approx 18.0$$

This up-weights the minority class in the loss so the model actually learns to find churners (high recall) rather than optimising raw accuracy.

### 3.3 No early stopping

`n_estimators=50` is fixed — there is **no early\_stopping\_rounds**. The holdout set is shown in `eval_set` **for monitoring only** (the learning-curve plot), and never influences training. (Adding early stopping on a validation slice is a possible future improvement.)

## 4. Data Pipeline & Feature Selection

## 4.1 Feature source and point-in-time integrity

Features come from `customer_features` (the feature store), one row per `(customer_id, as_of_date)`. Training uses two snapshot dates (`2026-07-17`, `2026-07-22`) and evaluates on a later holdout date (`2026-07-27`). Every feature is computed "as of" its date — **no future data leaks into the past**.

## 4.2 Leakage guard

A hardcoded `LEAKAGE_FEATURES` set is excluded **before** `model.fit()`:

```
days_since_last_txn, behav_recency_score, risk_dormant_indicator,
engagement_score, behav_inactive_days_90d, inactivity_streak_days,
behav_activity_consistency, txn_frequency_trend
```

plus the label column itself (`rel_customer_status`, pulled from config). These are recency/timing features that are effectively the definition of churn — using them would let the model "cheat".

## 4.3 Dead-feature scan (automatic)

Before training, the script scans the DB and auto-excludes any numeric column that is **all-zero** or **100% NULL** across the training dates (e.g. this run dropped `behav_txn_count_7d`, `temp_*_activity_ratio_90d`, and `prof_declared_vs_observed_income_ratio`).

## 4.4 Label construction

`generate_churn_label()` reads the configured status column and returns:

- `1` if `status == label_churn_positive_value` (e.g. "Closed")
- `0` if `status` is set but different
- `None` (row dropped) if status is missing/ambiguous

## 4.5 Result

```
9996 rows → 64 training features (9 leakage + 5 dead + 6 non-feature excluded)
Labels: 526 positive, 9470 negative (5.3%)
```

---

# 5. Validation & Evaluation Strategy

## 5.1 Holdout evaluation (temporal)

The model is scored on a **future** date it never saw. This mirrors production: train on history, score tomorrow.

## 5.2 Time-series cross-validation (out-of-fold)

To get an honest estimate **without** touching the holdout, the script runs 5-fold **time-series** CV over the training set (`TimeSeriesSplit`). Each fold trains on a temporal prefix and predicts the suffix. This respects

ordering — unlike random **KFold**, it cannot accidentally train on the future.

Because `cross_val_predict()` rejects **TimeSeriesSplit**, the fold loop is done manually with `sklearn.base.clone(model)`.

**Important finding:** time-series OOF AUC is **0.5504** vs holdout **0.8117**. The model ranks well on the specific holdout date but generalises poorly across the 5-day gap — the subject of the ongoing drift investigation.

## 5.3 Metrics

| Metric         | Formula                                                                    | What it tells us                                     |
|----------------|----------------------------------------------------------------------------|------------------------------------------------------|
| <b>AUC</b>     | Area under the ROC curve                                                   | Ranking: can we separate churners from non-churners? |
| <b>Brier</b>   | $\frac{1}{N} \sum_i (\hat{p}_i - y_i)^2$                                   | Mean squared probability error                       |
| <b>LogLoss</b> | $-\frac{1}{N} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$ | Calibration of confidence                            |
| <b>ECE</b>     | $\sum_b \frac{1}{N_b} \sum_{i \in b}  \hat{p}_i - y_i $                    | B <sub>b</sub>                                       |

## 6. Probability Calibration

### 6.1 Why raw probabilities are unreliable

XGBoost optimises **ranking**, not absolute probabilities. Raw scores were heavily distorted: **ECE = 0.40** (a predicted 0.7 meant "very likely churn", but the true rate was ~5%). For business decisions, probabilities must mean real rates.

### 6.2 Two calibration methods

**Platt scaling** — fits a logistic regression on the raw score  $s$ :

$$\hat{p} = \frac{1}{1 + \exp(-(a s + b))}$$

**Isotonic regression** — fits a monotonic, non-parametric step function that maps  $s \rightarrow \hat{p}$  with no assumed shape.

|            | Platt               | Isotonic                         |
|------------|---------------------|----------------------------------|
| Form       | Parametric sigmoid  | Non-parametric step function     |
| Data needs | Works on small data | Needs many samples (can overfit) |

### 6.3 Leakage-free fitting (out-of-fold)

The calibrator is fitted on **out-of-fold** predictions of the training set, never on the holdout. The method with the lowest holdout ECE wins.

This run: **Platt won** (ECE 0.0029 vs isotonic 0.0246) under time-series OOF.

## 6.4 At inference

`ChurnPredictor._apply_calibrator()` maps the raw score through the fitted transformer before returning `churn_probability`. Raw 0.7 → calibrated ≈ 0.066.

## 7. Diagnostics

| Diagnostic                 | What it detects                             | Current value                |
|----------------------------|---------------------------------------------|------------------------------|
| <b>Overfit gap</b>         | train AUC – test AUC (flag if > 0.03)       | 0.0435 (flag)                |
| <b>PSI drift</b>           | distribution shift per feature (see below)  | 9.51 → <b>freeze_scoring</b> |
| <b>Overconfidence</b>      | fraction of ≥0.9 predictions that are wrong | 0.0                          |
| <b>Confusion matrix</b>    | TP/FP/FN/TN at a threshold                  | 3031/1704/43/220 @ 0.5       |
| <b>Precision/Recall/F1</b> | trade-off at the operating point            | 0.114 / 0.837 / 0.201        |

### Population Stability Index (PSI):

$$\text{PSI} = \sum_b (\% \text{test}_b - \% \text{train}_b) \cdot \ln \frac{\% \text{test}_b}{\% \text{train}_b}$$

Interpretation: < 0.10 low, 0.10–0.25 moderate, > 0.25 **high** → **freeze scoring**.

### Decision thresholds:

- **Youden's J** ( $J = \text{TPR} - \text{FPR}$ ): the balanced point (0.4998 raw).
- **F1-max**: the threshold that maximises F1 (0.5683 raw) — better when false positives are expensive (e.g. personal calls to 1000s of non-churners).

## 8. Artifacts Produced

| Artifact                           | Path                                                             |
|------------------------------------|------------------------------------------------------------------|
| Trained model                      | <code>models/champion/churn/xgboost_churn_v1.json</code>         |
| Calibrator                         | <code>models/champion/churn/churn_calibrator_platt.joblib</code> |
| Registry (metadata)                | <code>models/registry.json</code>                                |
| Markdown report                    | <code>models/champion/churn/plots/training_report.md</code>      |
| <b>PDF report (plots embedded)</b> | <code>models/champion/churn/plots/training_report.pdf</code>     |
| Plots (8 PNGs)                     | <code>models/champion/churn/plots/</code>                        |

## 9. Reading the Plots

| Plot                                                                   | What to look for                                             |
|------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>roc_curve.png</code>                                             | Curve hugging the top-left = better ranking                  |
| <code>learning_curve.png</code>                                        | Train vs holdout AUC per round; a widening gap = overfitting |
| <code>calibration.png</code> / <code>calibration_comparison.png</code> | Points on the diagonal = calibrated                          |
| <code>feature_importance.png</code>                                    | Which features dominate the model                            |
| <code>confusion_matrix.png</code>                                      | FP vs FN balance at the chosen threshold                     |
| <code>prediction_distribution.png</code>                               | How separated the two classes' scores are                    |
| <code>threshold_analysis.png</code>                                    | Precision/recall/F1 trade-off vs threshold                   |

## 10. How to Retrain / Tune

```
.\.venv\Scripts\python.exe scripts\train_models.py
```

- **Improve ranking (AUC):** raise `max_depth` (4→5) or `n_estimators` (50→100), watch the overfit gap and time-series CV AUC.
- **Reduce overfit:** lower `max_depth`, raise `reg_lambda`/`min_child_weight`, reduce `colsample_bytree`.
- **Shift the operating point:** change `PRED_CHURN_THRESHOLD` on the **calibrated** scale ( $\approx 0.057$  Youden /  $\approx 0.059$  F1-max), not the raw scale.
- **Fix drift:** winsorise or quantile-bin the drifting features before retraining.

## 11. Key Numbers (current run)

| Metric                       | Value                                             |
|------------------------------|---------------------------------------------------|
| Holdout AUC                  | 0.8117                                            |
| Time-series OOF CV AUC       | 0.5504                                            |
| Brier / LogLoss (raw)        | 0.2115 / 0.6128                                   |
| Calibration (Platt) ECE      | 0.0029                                            |
| Features                     | 64                                                |
| Training rows / holdout rows | 9,996 / 4,998                                     |
| Class balance                | 5.3% positive, <code>scale_pos_weight</code> 18.0 |